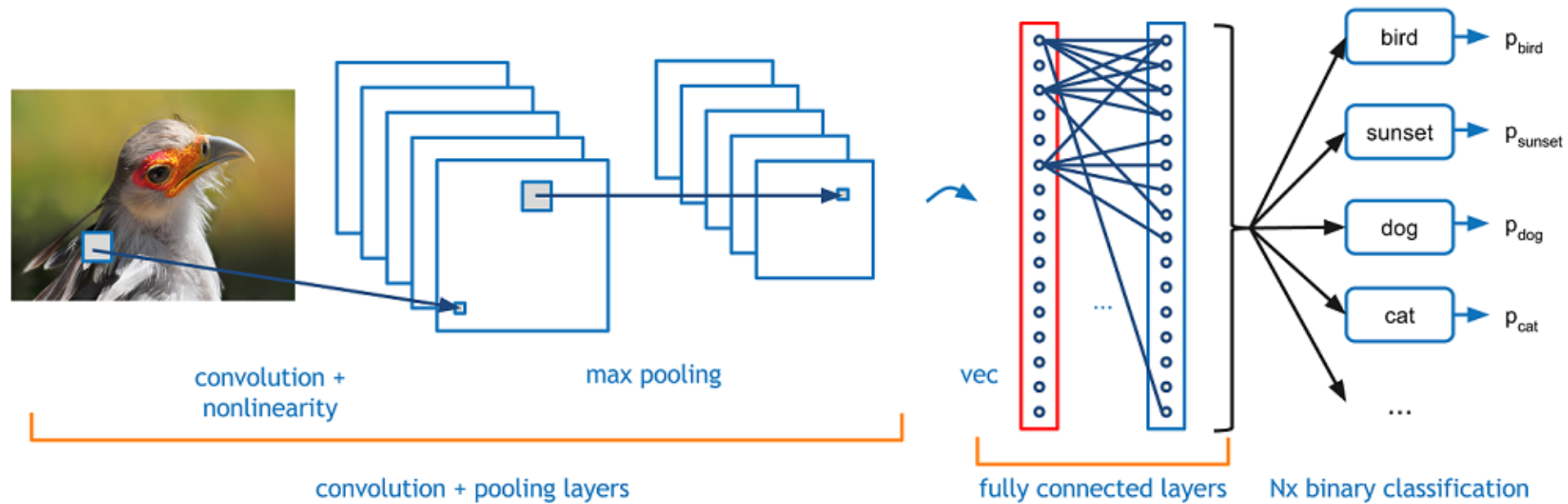


Inteligência Artificial

Visão Computacional: CNN, Redes Pré-treinadas e *Transfer Learning*

Prof. Dr. Ivan Carlos Alcântara de Oliveira

<https://orcid.org/0000-0002-6020-7535>



Visão Computacional

Introdução

- Visão computacional é uma **área da Inteligência Artificial** dedicada a fazer com que **máquinas possam enxergar, interpretar e agir com base em imagens e vídeos.**
- Os **humanos** utilizam a **visão** para **interagir com o ambiente**, os **sistemas** de visão computacional **visam replicar essa capacidade** por meio de algoritmos e redes neurais.
- Visão Computacional tem como **objetivo permitir que máquinas "vejam"**, interpretem e compreendam imagens e vídeos do mundo real, **simulando a visão humana.**

Visão Computacional

Aplicações

Algumas aplicações incluem:

- **Diagnóstico por imagens médicas** (radiografias, ultrassonografias, tomografias)
- Agricultura inteligente (**detecção de pragas em plantas**)
- **Veículos autônomos** (detecção de pista, pedestres, placas)
- Monitoramento de segurança (Circuito Fechado de Televisão (CFTV) com **reconhecimento facial**)
- Indústria (inspeção automatizada de qualidade)
- Florestas (detecção de incêndio)
- A popularização de **GPUs** e de **grandes bases de dados como ImageNet** permitiu **avanços consideráveis nos últimos anos.**

Visão Computacional

Desafios Técnicos e Teóricos - Resumo

Categoria	Exemplos de Desafios
Visual	Iluminação, ruído, perspectiva, oclusão, ambiguidade
Dados	Escassez de rótulos, desbalanceamento, variância
Generalização	Mudanças de domínio, contextos inéditos
Computacional	Processamento em tempo real, redes profundas e pesadas
Semântica/Contexto	Dificuldade em entender intenção, relações entre objetos
Ética e Privacidade	Viés algorítmico, invasão de privacidade, uso indevido

Convolutional Neural Networks

Conceito

- *Convolutional Neural Networks* (Redes Neurais Convolucionais - **CNNs**) são **redes que utilizam camadas convolucionais para extrair padrões locais de imagens (bordas, formas, texturas) que, ao longo das camadas, combinam essas informações simples para formar conceitos mais complexos e semânticos (rostos, objetos, etc.)**.
- Elas substituem camadas densas tradicionais por **operações convolucionais**, o que **reduz o número de parâmetros e melhora a capacidade de generalização**.

Convolutional Neural Networks

Conceito

Considerando as camadas de um CNN, temos:

- **Camadas iniciais:** detectam padrões simples (linhas, cantos, cores).
- **Camadas intermediárias:** combinam esses padrões em formas ou partes de objetos (olhos, rodas, folhas).
- **Camadas finais:** integram tudo em conceitos abstratos (um rosto humano, um carro, uma árvore).
- Essa “abstração crescente” é o que permite à CNN entender o conteúdo da imagem, e não apenas seus pixels.

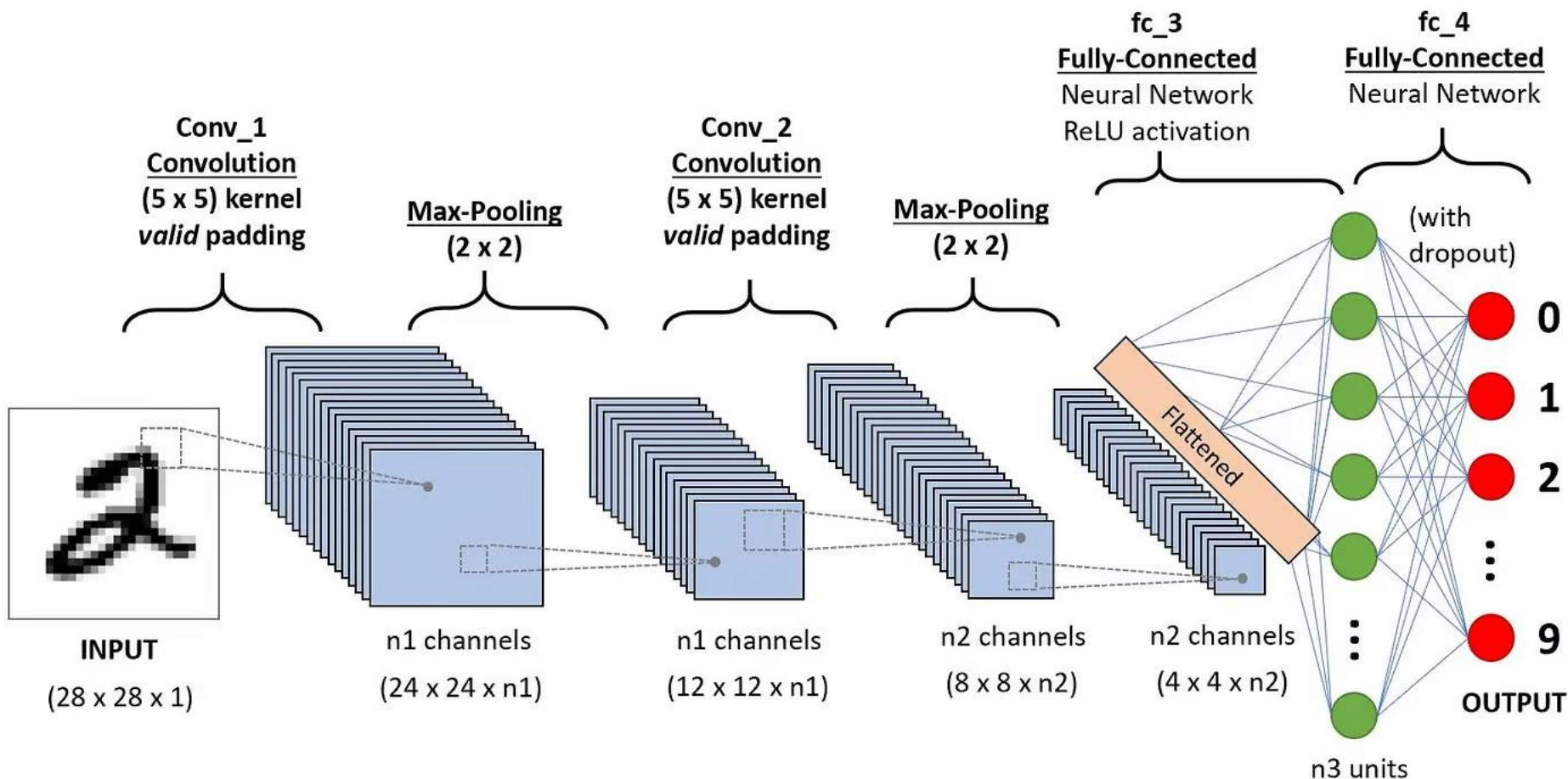
Convolutional Neural Networks

Conceito – Camada Convolutacional

- De forma simples, uma camada convolutacional é o “olho” da CNN.
- Ela desliza pequenos filtros (ou janelas) sobre a imagem, detectando padrões locais como bordas, linhas, curvas ou texturas.
- Cada filtro aprende um tipo diferente de característica, e as próximas camadas combinam esses padrões simples em formas mais complexas, permitindo que a rede “entenda” o conteúdo da imagem.

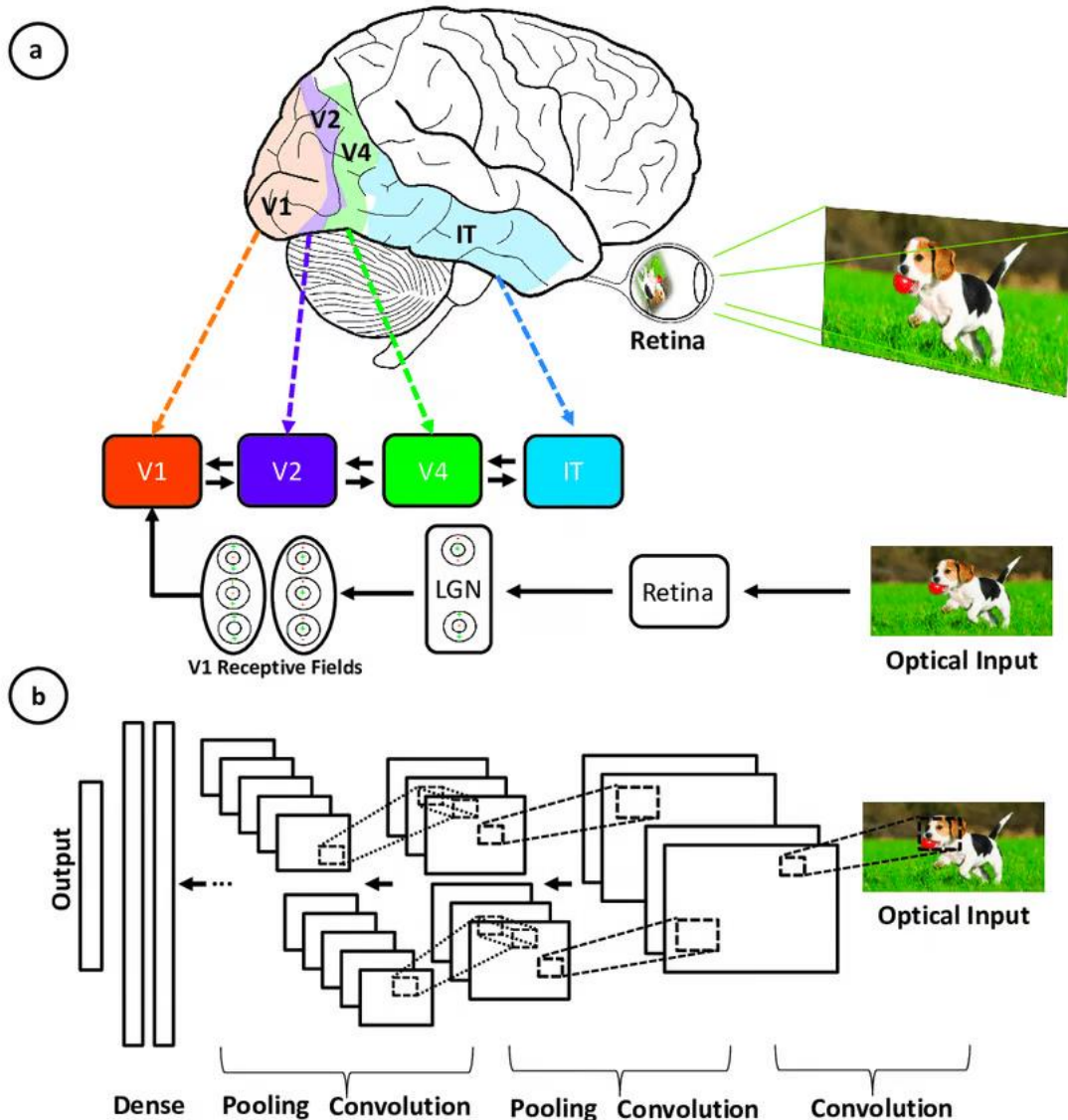
Convolutional Neural Networks

Arquitetura - Imagem



Convolutional Neural Networks

Arquitetura - Imagem



Correspondência entre as áreas associadas ao córtex visual primário e as camadas em uma rede neural convolucional.

Fonte: ROFFO, Giorgio. Ranking to learn and learning to rank: On the role of ranking in pattern recognition applications. arXiv preprint arXiv:1706.05933, 2017..

Convolutional Neural Networks

Correspondência entre as áreas associadas ao córtex visual primário e as camadas em uma rede neural convolucional

- As CNNs **imitam o sistema visual humano**, mas são mais simples, **sem seus complexos mecanismos de feedback e contando com o aprendizado supervisionado** em vez do não supervisionado **impulsionando avanços na visão computacional** apesar dessas diferenças.

Convolutional Neural Networks

Principais elementos

- ***Filtro/Kernels***: matrizes que **deslizam** sobre a imagem e **extraem características** específicas.
- ***Stride***: quantidade de **deslocamento do filtro**.
- ***Padding***: **adição de pixels extras** para preservar tamanho.
- ***Pooling***: operações para **reduzir a dimensão** (*MaxPooling*, *AveragePooling*).
- ***Flatten***: **transformar dados 2D em vetor 1D** para entrada na camada densa.
- **Função de ativação**: especialmente ReLU, tanh ou sigmoid, aplicadas após cada **convolução**.

Esses componentes **permitem que a CNN aprenda a detectar padrões importantes** sem extração manual de características.

Convolutional Neural Networks

Ordem de Estudo Sugerida

- 1. Arquitetura de CNNs:** tensores, camadas convolucionais, *pooling*, *fully connected*.
- 2. Processo de treinamento:** *loss*, otimização, validação.
- 3. Problemas comuns em redes profundas**
 - *Overfitting*
 - *Underfitting*
 - Generalização
- 4. Data Augmentation**
 - Ajuda a reduzir *overfitting* e a aumentar a robustez e diversidade dos dados de treino
- 5. Regularização** (*Dropout*, *BatchNorm*)
- 6. Treinamento por transferência** (*Transfer Learning*)
- 7. Fine-tuning**
- 8. Melhoria de desempenho** (*scheduler*, *tuning* de hiperparâmetros)

Convolutional Neural Networks

Tensores - Conceitos

- Um tensor é uma função multilinear que mapeia vetores para um número, mas na prática computacional podemos entendê-lo como **uma matriz representada por um *array* multidimensional**.
- Imagine um tensor como **uma caixa de números organizados em múltiplos eixos**, onde cada eixo representa uma propriedade dos dados:
 - 1º eixo → número de amostras (batch)
 - 2º eixo → canais (ex: RGB – Red, Green, Blue)
 - 3º e 4º → altura e largura (imagem)

Convolutional Neural Networks

Tensores – Comparação com estruturas conhecidas

Nome	Dimensão	Exemplo	Nome técnico
Escalar	0D	$x = 7$	Tensor de ordem 0
Vetor	1D	$x = [7, 3, 1]$	Tensor de ordem 1
Matriz	2D	$x = [[1, 2], [3, 4]]$	Tensor de ordem 2
Cubo	3D	Conjunto de imagens RGB	Tensor de ordem 3
Lote de imagens	4D	Conjunto de imagens com canais	Tensor de ordem 4

Convolutional Neural Networks

Tensores – Como tensores representam dados visuais em CNNs

Nível	Interpretação prática	Exemplo
1 imagem RGB	Tensor 3D: [C, H, W]	[3, 224, 224]
Lote de imagens	Tensor 4D: [B, C, H, W]	[32, 3, 224, 224]
Mapa de ativações	Saída de uma convolução	[32, 64, 112, 112]
Vetor de características	Saída de <i>flatten/fully connected</i>	[32, 512]

Letra	Nome técnico	Significado prático
B	<i>Batch Size</i>	Número de imagens no lote (quantas imagens são processadas de uma vez)
C	<i>Channels</i>	Número de canais de cor (ex: 3 para RGB, 1 para grayscale, 4 para RGBA)
H	<i>Height</i> (Altura)	Número de pixels na vertical
W	<i>Width</i> (Largura)	Número de pixels na horizontal

Convolutional Neural Networks

Tensores – Batch Size

Durante o treinamento de uma rede neural, o modelo:

- Recebe um conjunto de dados de entrada (imagens, textos, etc.).
- Calcula a **saída** para esse conjunto.
- Mede o **erro (loss)** comparando com o valor esperado.
- Ajusta os **pesos** para reduzir esse erro.

Processar todo o dataset de uma vez pode ser inviável (por consumo de memória e tempo).

Por isso, o conjunto de dados é dividido em **lotes (batches)**.

Convolutional Neural Networks

Tensores – Batch Size - Exemplo

Imagine um dataset com **10.000 imagens** e `batch_size = 100`.

- Isso significa que, a cada iteração, a CNN processará **100 imagens**.
- Depois de calcular a perda média para essas 100 imagens, fará **uma atualização dos pesos**.
- Para processar todas as 10.000 imagens, serão necessárias:
 $10.000 / 100 = 100$ iterações, então, isso forma **uma época (epoch)**.

Convolutional Neural Networks

Tensores – Operações comuns com Tensores em CNN

- **Indexação:** acessar partes do tensor.
- **Permute/transpor:** trocar eixos (importante ao alternar entre Keras e PyTorch).
- **Reshape (view):** converter de $[B, C, H, W]$ para $[B, -1]$ antes da *fully connected*.
- **Broadcasting:** técnica utilizada por frameworks como NumPy e PyTorch para **realizar operações entre tensores de formas (shapes) diferentes**, sem precisar replicar manualmente os dados menores.

Convolutional Neural Networks

Tensores – Por que tensores são úteis em aprendizado profundo?

- **Permitem representar dados estruturados complexos.**
- São compatíveis com operações vetorizadas **altamente paralelizáveis.**
- **Facilitam o uso de GPU (*Graphics Processing Unit*)/TPU (*Tensor Processing Unit*) para acelerar treinamento.**
- Compatíveis com autodiferenciação (gradientes automáticos).

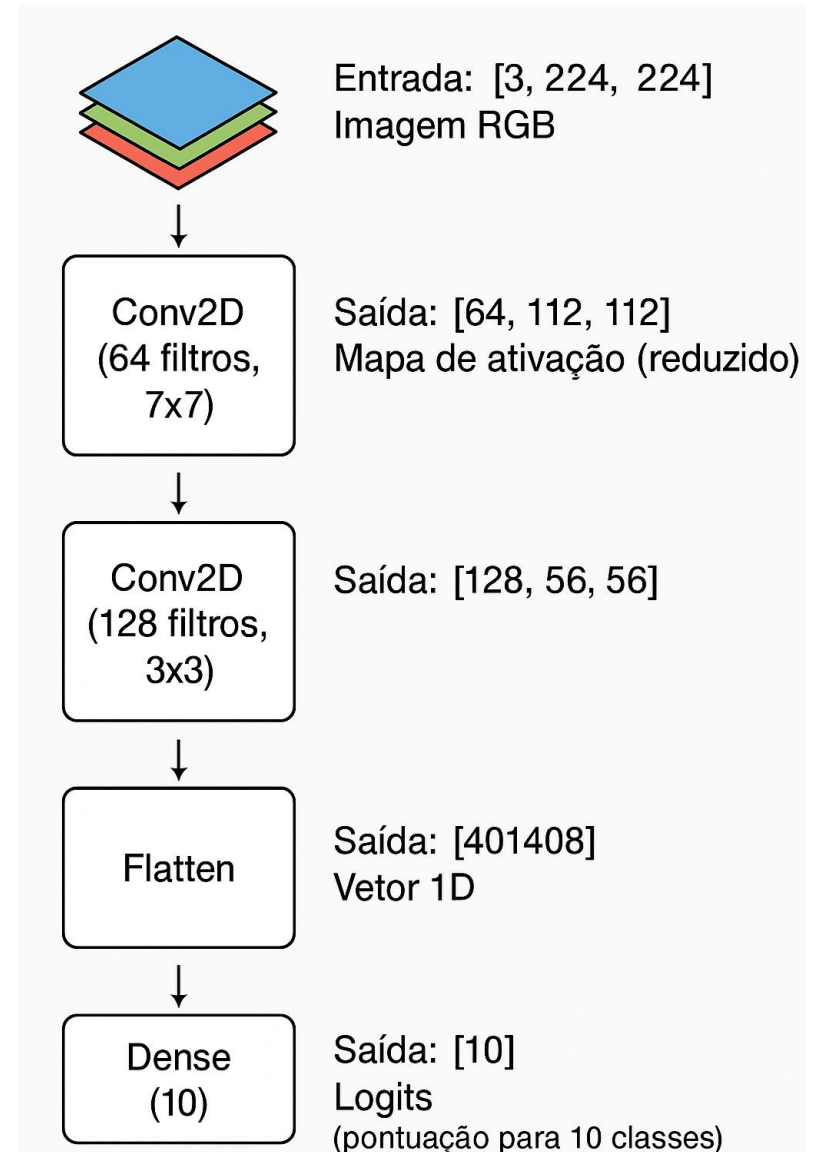
Convolutional Neural Networks

Tensores – Ilustração da evolução de um tensor em CNN - imagem 224x224x3
– Diagrama Visual

Exemplo: Se a entrada tiver 3 canais (RGB) e a camada tiver 64 filtros, o resultado será um tensor com 64 canais.

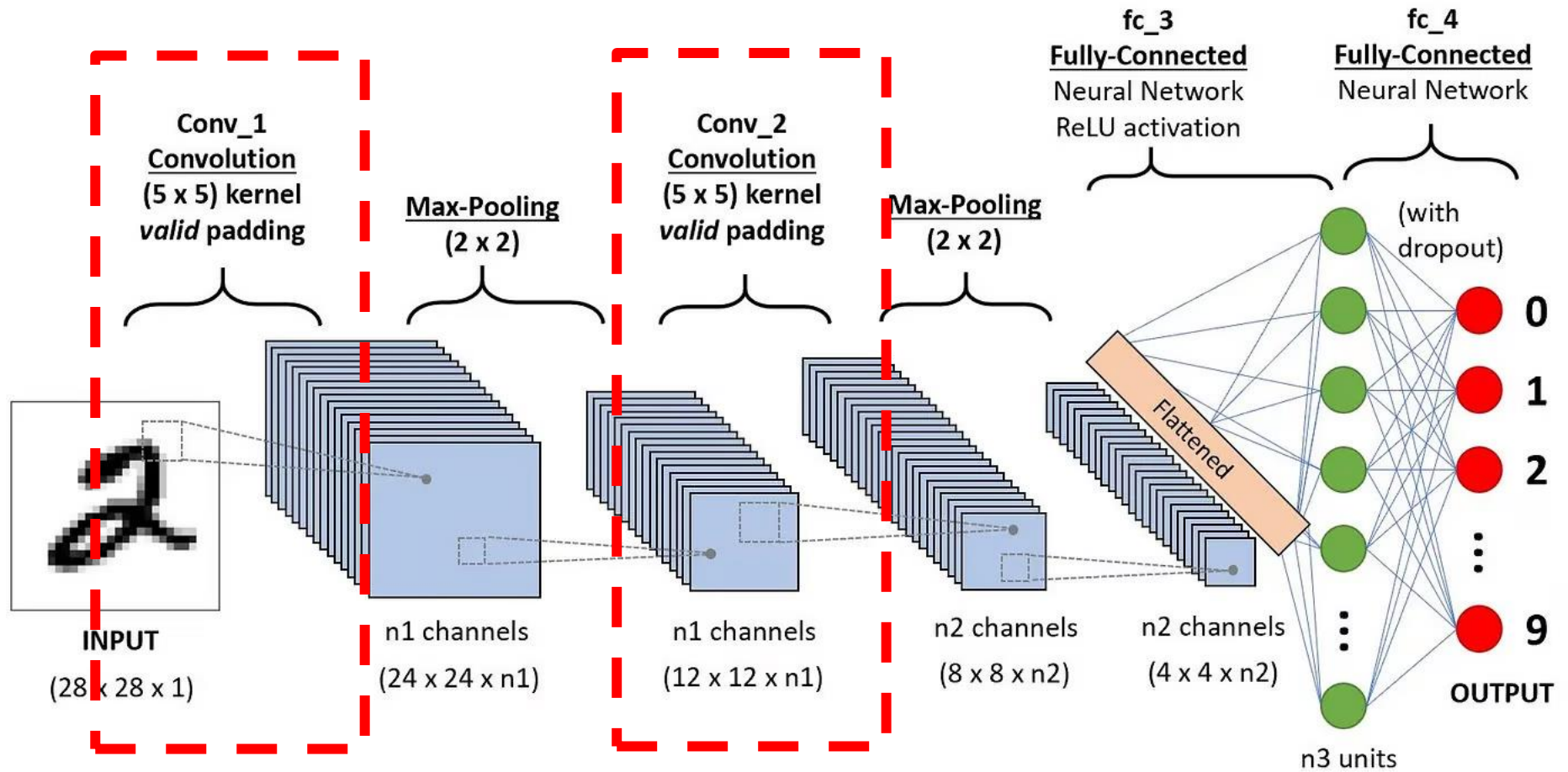
Cada um dos 64 filtros aprende a detectar um tipo de padrão diferente na imagem, como:

- bordas horizontais ou verticais,
- cantos, curvas ou texturas,
- contraste entre claro e escuro,
- ou até partes específicas de objetos (como olhos, folhas, rodas etc.) nas camadas mais profundas.



Convolutional Neural Networks

Camada Convolutacional



Convolutional Neural Networks

Camada Convolutacional

- É o elemento central de uma CNN, **projetada para extrair automaticamente características (ou *features*) espaciais e locais de entradas estruturadas, como imagens, áudio ou dados em grid.**
- **Realiza uma operação matemática chamada convolução, que aplica filtros (ou kernels) deslizando sobre a entrada** (por exemplo, uma imagem) para detectar padrões locais, como bordas, texturas, formas, etc.

Convolutional Neural Networks

Camada Convolutacional - Conceitualmente

- **É como uma janela deslizando (o filtro) que percorre a imagem e realiza operações ponto a ponto (multiplicação + soma) entre os valores do filtro e da região da imagem que está sendo analisada.**
- **Isso gera um mapa de ativação ou mapa de características (*feature map*) que indica a presença daquele padrão na imagem.**

Convolutional Neural Networks

Camada Convolutacional - O que a camada convolutacional aprende?

- **Durante o treinamento**, os valores dos filtros são ajustados via backpropagation para **detectar características úteis para a tarefa** (como **classificação**).
- Nos **primeiros níveis**, a rede **pode aprender**:
 - **Bordas horizontais/verticais**
 - **Cantos**
 - **Texturas**
- Nas **camadas mais profundas**, ela **aprende**:
 - **Formas complexas**
 - **Partes de objetos**
 - **Composições semânticas**

Convolutional Neural Networks

Camada Convolutacional – O que ocorre?

- A cada passo (posição do filtro), o valor na saída deve ser:

$$\text{Output}(i, j) = \sum_{m, n} \text{Input}(i + m, j + n) \times \text{Filtro}(m, n)$$

- Ou seja, **se a janela do filtro (K) passa por regiões com valores diferentes de zero, os resultados devem mudar** conforme o conteúdo da entrada muda.
- Quando uma CNN aplica um filtro sobre a imagem, ela multiplica os valores da janela local da imagem pelos valores do filtro e soma os resultados – exatamente como a correlação cruzada.
- O que chamamos de “**convolução**” nas CNNs, **matematicamente é chamado correlação cruzada.**

Convolutional Neural Networks

Camada Convolucional – O que ocorre? - Exemplo

- Considere: Entrada (Input 4x4):

1	1	0	0
0	1	1	0
0	0	1	1
0	0	0	1

- Filtro (Kernel 2x2):

1	0
0	1

- Operação: Na primeira posição (canto superior esquerdo):

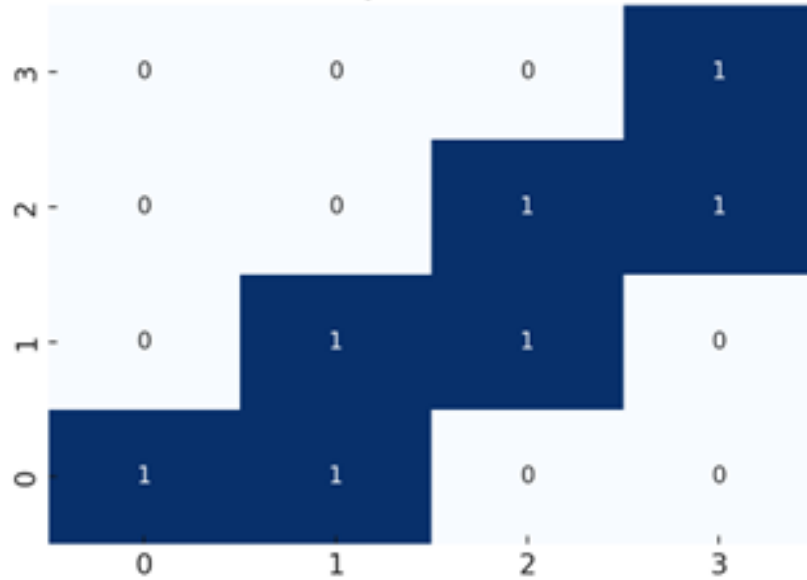
$$1 \times 1 + 1 \times 0 + 0 \times 0 + 1 \times 1 = 1 + 0 + 0 + 1 = 2$$

- Se mover o filtro uma célula para a direita, os valores multiplicados mudam, logo o valor na saída também deve mudar.

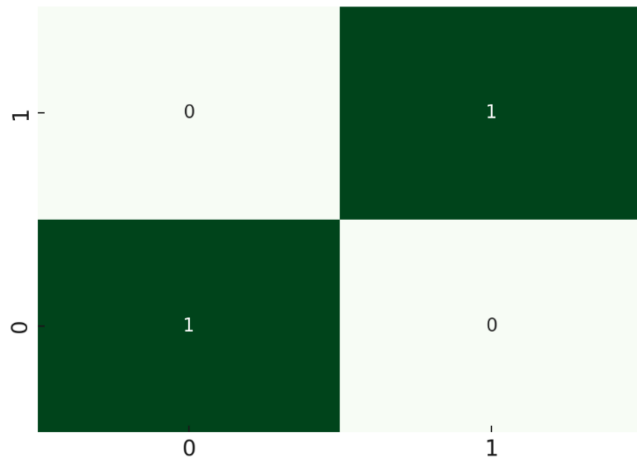
Convolutional Neural Networks

Camada Convolutacional – O que ocorre? – Exemplo

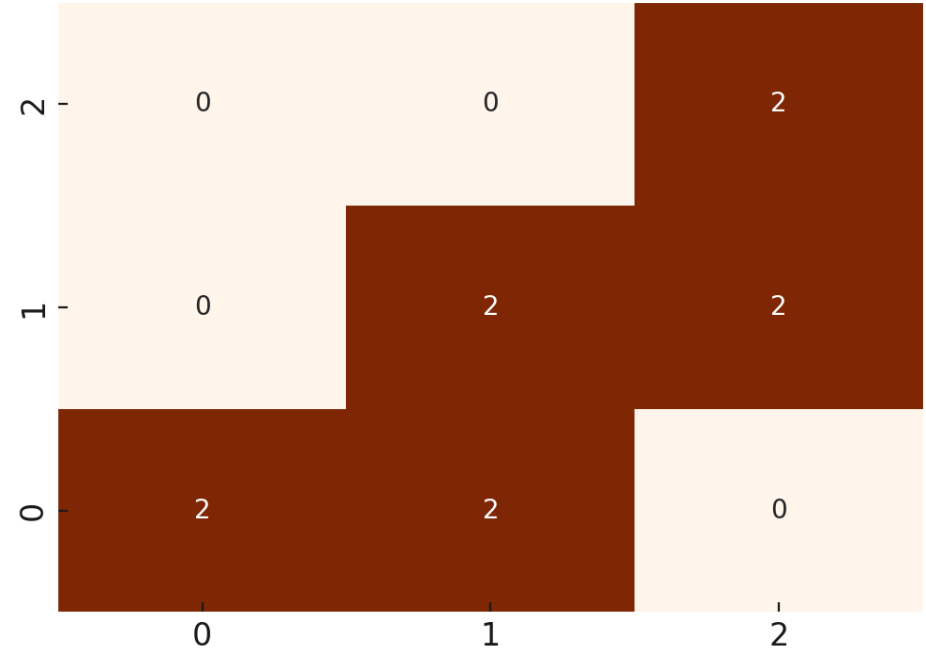
Input (4x4)



Filter (2x2)

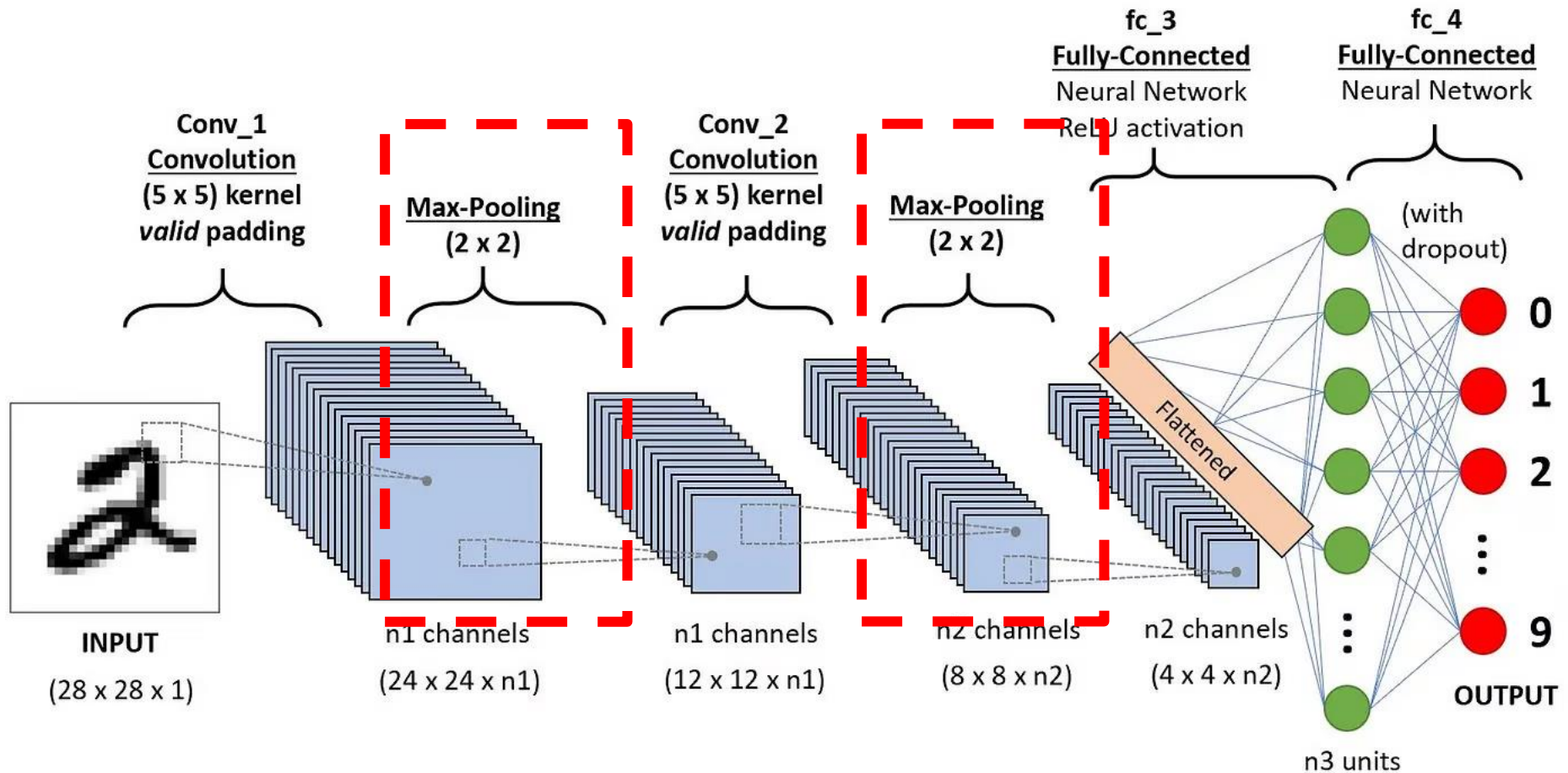


Output (Feature Map)



Convolutional Neural Networks

Camadas - Camada de *Pooling* (Subamostragem)



Convolutional Neural Networks

Camadas - Camada de *Pooling* (Subamostragem)

Função:

- **Reduz a dimensão espacial dos mapas de ativação.**
- Ajuda a criar invariância a pequenas variações (ex: translações).
- **Diminui o número de parâmetros e custo computacional.**
- Funciona como um “**resumo local**” das informações — pega o que há de mais relevante em pequenas regiões da imagem.

Como funciona:

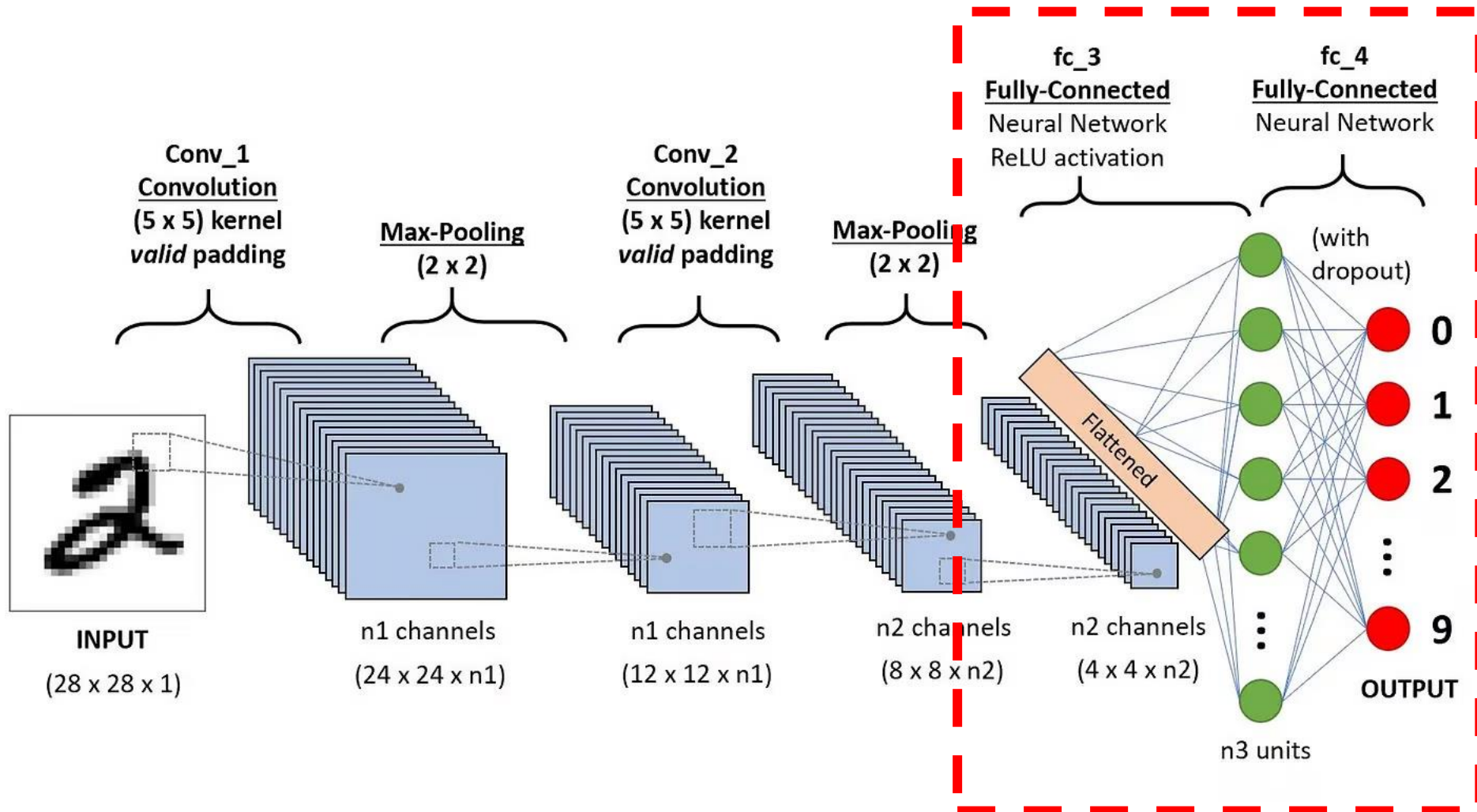
- Usa uma janela de *pooling* (ex.: 2x2) que percorre a imagem com *stride*.

Tipos comuns:

- ***Max Pooling***: retorna o valor **máximo** da janela.
- ***Average Pooling***: retorna a **média** da janela.

Convolutional Neural Networks

Camadas - Camada Totalmente Conectada (*Fully Connected / Dense Layer*)



Convolutional Neural Networks

Camadas - Camada Totalmente Conectada (*Fully Connected / Dense Layer*)

Função:

- **Realiza interpretação global das características extraídas.**
- **Fornece a saída final**, geralmente uma classificação ou valor contínuo.

Como funciona:

- **Cada neurônio é conectado a todos os neurônios da camada anterior.**
- **A entrada/saída geralmente é um vetor 1D** (é preciso achatar a saída convolucional/*pooling*).

Convolutional Neural Networks

Camadas - Síntese

- A camada **Convolutional** funciona como um conjunto de “lupas” que passam pela imagem procurando padrões.
 - Ela identifica coisas como bordas, cores e formas simples.
Com isso, a rede começa a entender o que existe na imagem, como olhos, rodas ou objetos.
- A camada de **Pooling** “resume” a imagem, diminuindo seu tamanho.
 - Ela guarda as partes mais importantes, como bordas, formas e padrões.
Assim, a CNN fica mais rápida e consegue focar no que realmente ajuda a reconhecer a imagem.
- A camada **Fully Connected (Dense)** é como a parte final que “toma a decisão”.
 - Ela junta tudo o que a rede aprendeu sobre a imagem.
E então decide qual é o objeto, como dizer “isso é um gato” ou “isso é um carro”.

Convolutional Neural Networks

Arquiteturas Recentes e Modernas - Resumo

Arquitetura	Ano	Inovação-chave	Parâmetros (ex.)	Desempenho
LeNet-5	1998	CNN funcional com pooling	60k	Baixo
AlexNet	2012	ReLU, dropout, 2 GPUs	60M	Alto
VGGNet	2014	Conv 3x3, arquitetura profunda	138M	Alto
GoogLeNet	2014	Módulo Inception	6.8M	Alto
ResNet (ResNet-50)	2015	Conexões residuais (skip)	25M	Muito Alto
DenseNet	2017	Conexões densas entre camadas	~8M	Muito Alto
Xception	2017	Depthwise Separable Conv	23M	Muito Alto
MobileNet(V1,V2,V3)	2017	leves para dispositivos móveis	~3.4M	Muito Alto
RepVGG (A0 a B1g4)	2021	Reparametrização estrutural (classificação em tempo real)	~60.2 M	Muito Alto
EfficientNet-B0/V2	2019/ 2021	Escalonamento composto	5.3M	Estado da arte
ViT	2020	Transformer puro para imagens	86M+	Estado da arte
ConvNeXt/V2	2022/ 2023	CNN moderna redesenhada	89M	Estado da arte

Convolutional Neural Networks

Arquiteturas Clássicas, Recentes e Modernas – Resumo com Frameworks

Arquitetura	Alguns Frameworks Disponíveis
LeNet-5	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, Timm
AlexNet	PyTorch, ONNX, TensorFlow Lite, Timm
VGGNet	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, Hugging Face Transformers, Timm
GoogLeNet	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, Timm
ResNet	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, JAX / Flax, Hugging Face Transformers, Timm
DenseNet	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, Hugging Face Transformers, Timm
Xception	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, Google Cloud AutoML
MobileNetV2	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, TFLITE Micro
RepVGG	PyTorch, TensorFlow, ONMX, NVIDIA Tensor RT
EfficientNet-B0/V2	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, JAX / Flax
ViT	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, JAX / Flax, Hugging Face Transformers
ConvNeXt/V2	TensorFlow / Keras, PyTorch, ONNX, TensorFlow Lite, Hugging Face Transformers

Convolutional Neural Networks

Data Augmentation - Conceitos

- É o processo de gerar novas amostras de dados a partir das existentes, aplicando transformações controladas que mantêm o significado semântico da imagem.
- *Data Augmentation* ensina a CNN a reconhecer o mesmo objeto em diferentes perspectivas, posições, iluminações e condições, ampliando seu aprendizado e evitando a "memorização" das imagens originais.
- Por exemplo:
 - Uma CNN treinada apenas com imagens de gatos em fundo branco pode falhar ao encontrar gatos em fundo escuro ou de lado e *data augmentation* resolve isso.

Convolutional Neural Networks

Data Augmentation - Características

Característica	Descrição
 Expansão do dataset	Aumenta a quantidade de exemplos de forma artificial
 Diversidade	Introduz variações nos dados sem mudar a classe
 Estocástico	Transformações aplicadas aleatoriamente durante o treinamento
 Indução de Robustez	Ensina o modelo a ignorar variações irrelevantes
 Genérico e Independe da Arquitetura	Funciona com qualquer rede CNN, incluindo redes pré-treinadas
 Sem aumento de rótulo	Não altera os rótulos associados às imagens

Convolutional Neural Networks

Data Augmentation – Técnicas comuns e exemplos

Técnica	Efeito	Exemplo prático
Rotação (<i>Rotation</i>)	Simula variação de orientação	Imagens de plantas inclinadas
<i>Flip</i> horizontal	Espelhamento horizontal	Rosto virado
<i>Zoom</i>	Aumentar ou reduzir o tamanho do objeto	Captura mais próxima ou distante
<i>Brightness/Contrast</i>	Simula variação de iluminação	Imagens externas (dia/noite)
<i>Crop</i>	Recorte parcial da imagem	Enfatizar detalhes
Ruído gaussiano	Simula imperfeições de câmera	Imagens com baixa qualidade
MixUp / CutMix	Mistura de duas imagens com rótulo médio	Aumenta robustez a ambiguidades

Convolutional Neural Networks

Regularização (Dropout, BatchNorm) - Dropout

- A regularização é um conjunto de técnicas usadas para evitar *overfitting*, ou seja, impedir que o modelo aprenda demais os detalhes e ruídos do conjunto de treinamento.

Técnicas:

- Dropout é uma técnica de regularização que **desativa (zera) aleatoriamente uma parte dos neurônios durante o treinamento**. Normalmente usada após as camadas densas (*fully connected*)

```
model.add(layers.Dense(128, activation='relu'))  
model.add(layers.Dropout(0.5))  # 50% dos neurônios  
desligados no treino
```

Convolutional Neural Networks

Transfer Learning e Redes Pré-Treinadas

- Redes pré-treinadas são **modelos de redes neurais já treinados previamente em um grande conjunto de dados**, como o ImageNet12 (com mais de 1 milhão de imagens e 1000 classes).
- **Esses modelos aprenderam representações visuais genéricas**, úteis para várias tarefas de visão computacional.
- **Exemplos:** ResNet50, VGG16, MobileNet, EfficientNet etc.

Convolutional Neural Networks

Transfer Learning

- Tradicionalmente, os modelos de **aprendizado profundo exigem grandes volumes de dados e poder computacional** para alcançar alto desempenho.
- Em **muitas aplicações reais**:
 - Os **dados** disponíveis são **limitados**.
 - O **custo de coletar e rotular** dados é **elevado**.
 - O **treinamento completo** é **demorado** e **computacionalmente caro**.
- É uma **técnica** de aprendizado de máquina onde o **conhecimento adquirido por um modelo ao resolver uma tarefa é reutilizado em uma segunda tarefa relacionada**.
- Em vez de treinar um modelo do zero, utiliza-se um modelo **previamente treinado para resolver um problema diferente, normalmente com uma base de dados menor e mais específica**.

Convolutional Neural Networks

Transfer Learning – Exemplo: Como Funciona em Visão Computacional

- Um modelo como o **ResNet-50 é treinado no ImageNet** (~21.841 classes - 14 milhões de imagens rotuladas. Pode ultrapassar 1 TB de dados),.
- **Esse modelo aprende características genéricas** de imagens: bordas, formas, texturas, etc.
- Para uma **nova tarefa** (por exemplo, classificar tipos de flores), **aproveita-se essa base e ajusta-se** (*fine-tune*) as **camadas finais para a nova tarefa**.

Convolutional Neural Networks

Transfer Learning – Estratégias

- 1. *Feature Extraction*** (Extração de Características): **todas as camadas convolucionais congeladas e substitui-se apenas as últimas camadas para a tarefa-alvo.**
- 2. *Fine-Tuning*** (Ajuste Fino): **algumas ou todas as camadas do modelo pré-treinado são descongeladas e ajustadas**
- 3. *Full Model Retraining*** (Retreinamento completo do modelo): **modelo inteiro é usado como ponto de partida e retreinado por completo com uma nova base de dados.**

Convolutional Neural Networks

Transfer Learning – Considerações

- ***Transfer learning* é recomendado** quando:
 - Tem **poucos dados** ou dados difíceis de rotular.
 - Quer **acelerar o treinamento**.
 - Quer **aproveitar representações gerais** aprendidas em grandes *datasets* como o ImageNet.
- **Treinar do zero é viável** quando:
 - O seu domínio de dados é **muito diferente** (ex.: imagens médicas, satélite).
 - Tem **muitos dados rotulados**.
 - Quer **estudar a rede desde a inicialização aleatória**.

Convolutional Neural Networks

Fine-Tuning – O que é? e Motivação

- ***Fine-Tuning* (ou Ajuste Fino) é uma técnica de *Transfer Learning*, onde se ajusta parte das camadas de um modelo pré-treinado para uma nova tarefa, especialmente útil quando há dados limitados para treinar uma CNN do zero.**

Situação comum	Por que usar <i>Fine-Tuning</i> ?
Poucos dados na nova tarefa	Reaproveita aprendizado anterior
Tarefa similar ao pré-treinamento	Pequenos ajustes bastam
CNN muito profunda ou complexa	Evita treinar do zero, reduz tempo
Quer maximizar desempenho com custo baixo	Ajusta apenas parte do modelo

Convolutional Neural Networks

Fine-Tuning – Etapas Clássicas

1. Carregar o modelo pré-treinado (ex.: com pesos do ImageNet)
2. Congelar as camadas convolucionais
3. Adicionar nova cabeça de classificação
4. Treinar a nova cabeça (*feature extraction*)
5. Descongelar parte da base convolucional
6. Treinar novamente (*fine-tune*)

Convolutional Neural Networks

Exemplo – PyTorch – Forma de fazer o *Split*

Framework	Split automático?	Como fazer o split
TensorFlow/TFDS	Sim	<code>split=['train[:70%]', 'train[70%:85%]', 'train[85%:]']</code> ,
PyTorch	Não	Manual com pastas ou <code>random_split()</code>

Convolutional Neural Networks

Exemplo - Objetivo

- O objetivo deste projeto é **desenvolver e comparar dois modelos de classificação de flores utilizando *Transfer Learning* com CNNs, empregando PyTorch (com ResNet18).**
- A proposta visa demonstrar:
 - Como reutilizar redes pré-treinadas em tarefas personalizadas;
 - A viabilidade de treinar modelos eficientes mesmo com poucos dados;
 - As diferenças práticas entre os frameworks adotados;
 - A aplicação prática da técnica para classificação de 5 espécies de flores com imagens.

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - Importação de Bibliotecas

```
import os
import shutil
import random
import torch
import torchvision
from torchvision import transforms, models
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
from tqdm import tqdm
```

Explicação:

- Foram usadas as bibliotecas do PyTorch e torchvision para carregar modelos e aplicar transformações.

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - Detecta GPU

```
#  Detecta GPU (ou usa CPU se indisponível)
```

```
device = torch.device("cuda" if  
torch.cuda.is_available() else "cpu")  
print("Dispositivo em uso:", device)
```

Explicação:

- Detecta a GPU ou usa a CPU, caso ela esteja indisponível.

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - Transformações para imagens

 Data augmentation e pré-processamento

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor()
])

val_test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor()
])
```

Explicação:

- As imagens precisam ser redimensionadas e normalizadas com os mesmos parâmetros do ImageNet, onde a ResNet18 foi treinada

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – Carregamento do Dataset

```
# 📁 Dataset deve estar previamente dividido em train/val/test  
(por pastas)  
data_dir = "tf_flowers_split"
```

Explicação:

- Identifica o dataset a ser utilizado, porém, as pastas devem estar adequadamente divididas para uso em train, val e test..

Convolutional Neural Networks

Dataset – Divisão do conjunto de Imagens

- Considere todas as imagens em uma única pasta por classe (ex: dataset/class1, dataset/class2).
- Estrutura inicial (original) esperada da pasta única contendo o dataset de imagens:

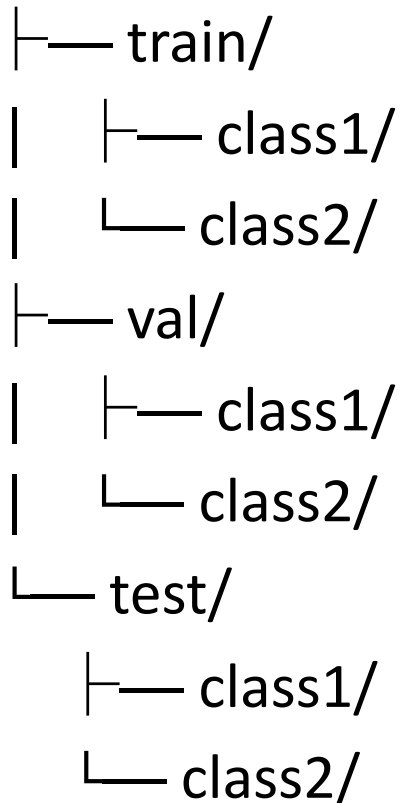
```
dataset_original/  
├── class1/  
│   ├── img1.jpg  
│   ├── img2.jpg  
├── class2/  
│   ├── img1.jpg  
│   └── img2.jpg
```

Convolutional Neural Networks

Dataset – Divisão do conjunto de Imagens - script

- A partir dos dados originais, eles devem ser separados em (treino) train/, (validação) val/ e (teste) test/, como ilustrado abaixo:

dataset_dividido/



Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - Carregamento de dados com ImageFolder.

```
#  Carregamento dos dados com as transformações
train_ds = torchvision.datasets.ImageFolder(os.path.join(data_dir, "train"),
transform=train_transform)
val_ds = torchvision.datasets.ImageFolder(os.path.join(data_dir, "val"),
transform=val_test_transform)
test_ds = torchvision.datasets.ImageFolder(os.path.join(data_dir, "test"),
transform=val_test_transform)

#  DataLoaders com pré-busca e threads paralelas
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True, num_workers=2,
pin_memory=True)
val_loader = DataLoader(val_ds, batch_size=32, shuffle=False, num_workers=2,
pin_memory=True)
test_loader = DataLoader(test_ds, batch_size=32, shuffle=False, num_workers=2,
pin_memory=True)
```

Explicação:

- Carregamento dos dados com as transformações e dataloaders com pré-busca e threads paralelas.

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - Modelo pré-treinado
ResNet18

```
# 📖 Modelo pré-treinado (ResNet18)
model = models.resnet18(weights=ResNet18_Weights.DEFAULT)

# ❄️ Congela a base convolucional
for param in model.parameters():
    param.requires_grad = False

# 🔄 Substitui a camada final (head) para 5 classes
model.fc = torch.nn.Linear(model.fc.in_features, 5)
model = model.to(device)
```

Explicação:

- Congela a base da ResNet18 e substitui a camada final para 5 classes.

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - Compilação e treinamento

```
#  Função de perda e otimizador
criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.fc.parameters(), lr=0.001)

#  Treinamento (Feature Extraction)
train_losses, val_losses = [], []
train_accs, val_accs = [], []
```

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - Compilação e treinamento

```
for epoch in range(6):
    model.train()
    total_loss, correct = 0, 0
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        out = model(x)
        loss = criterion(out, y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
        correct += (out.argmax(1) == y).sum().item()
    train_losses.append(total_loss / len(train_loader))
    train_accs.append(correct / len(train_ds))
```

Realiza o treinamento para as 5 classes

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - Compilação e treinamento

Validação

```
model.eval()
```

```
with torch.no_grad():
```

```
    val_loss, correct = 0, 0
```

```
    for x, y in val_loader:
```

```
        x, y = x.to(device), y.to(device)
```

```
        out = model(x)
```

```
        val_loss += criterion(out, y).item()
```

```
        correct += (out.argmax(1) == y).sum().item()
```

```
val_losses.append(val_loss / len(val_loader))
```

```
val_accs.append(correct / len(val_ds))
```

Realiza a validação para as 5 classes

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - *Fine-tuning*:
descongela toda a base

```
#  Fine-tuning: descongela toda a rede
```

```
for param in model.parameters():  
    param.requires_grad = True
```

```
#  Otimizador com taxa de aprendizado menor
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

Indicação do fine-tuning
descongelando toda a
rede e fazendo uso do
Adam como otimizador

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 - *Fine-tuning*:
descongela toda a base

🧠 Treinamento adicional com rede completa

```
for epoch in range(3):  
    model.train()  
    for x, y in train_loader:  
        x, y = x.to(device), y.to(device)  
        optimizer.zero_grad()  
        out = model(x)  
        loss = criterion(out, y)  
        loss.backward()  
        optimizer.step()
```

O fine-tuning é feito com uma taxa de aprendizado menor para ajustar toda a rede, incluindo a base.

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – *Avaliação Final no conjunto de teste*

```
#  Avaliação final no conjunto de teste
model.eval()
correct = 0
test_loss = 0
with torch.no_grad():
    for x, y in test_loader:
        x, y = x.to(device), y.to(device)
        out = model(x)
        test_loss += criterion(out, y).item()
        correct += (out.argmax(1) == y).sum().item()
test_loss /= len(test_loader)
test_acc = correct / len(test_ds)
print(f"Acurácia no conjunto de teste: {test_acc:.4f}")
```

Avaliação final no
conjunto de teste

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – Gráficos

 Gráficos

Apresenta os gráficos

```
plt.figure()
plt.plot(train_losses, label="Train Loss")
plt.plot(val_losses, label="Val Loss")
plt.axhline(y=test_loss, color='r', linestyle='--', label=f"Test Loss =
{test_loss:.4f}")
plt.legend()
plt.title("Loss por época com Teste")
plt.show()
```

```
plt.figure()
plt.plot(train_accs, label="Train Acc")
plt.plot(val_accs, label="Val Acc")
plt.axhline(y=test_acc, color='r', linestyle='--', label=f"Test Acc =
{test_acc:.4f}")
plt.legend()
plt.title("Acurácia por época com Teste")
plt.show()
```

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – *Obter Previsões no Conjunto de Teste*

```
import torch

model.eval()  # Colocar o modelo em modo de avaliação
y_true = []
y_pred = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(predicted.cpu().numpy())
```

Previsões no conjunto de testes

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – *Calcular e Exibir a Matriz de Confusão*

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Calcular a matriz de confusão
cm = confusion_matrix(y_true, y_pred)

# Exibir a matriz de confusão
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap=plt.cm.Blues)
plt.title("Matriz de Confusão - PyTorch")
plt.show()
```

Calcula e Exibe a matriz de confusão

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – *Salvar Modelo Gerado, Carregar e Teste com imagem nova*

```
# Salvando o modelo
model_path = "resnet18_model.pth"
torch.save(model.state_dict(), model_path)
print(f"Modelo salvo em {model_path}")

# Carregar uma nova imagem
from PIL import Image
import torch.nn.functional as F
```

Salva o modelo gerado com a ResNet18

E import para carregar a imagem

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – *Salvar Modelo*

Gerado, Carregar e Teste com imagem nova

```
def predict_new_image(image_path, model, device, transform):  
    # Carregar e pré-processar a imagem  
    image = Image.open(image_path)  
    image = transform(image).unsqueeze(0) # Adiciona uma dimensão extra para o  
batch  
    image = image.to(device)  
  
    # Colocar o modelo em modo de avaliação  
    model.eval()  
  
    # Fazer a previsão  
    with torch.no_grad():  
        output = model(image)  
        probabilities = F.softmax(output, dim=1) # Calcular as probabilidades  
        confidence, predicted = torch.max(probabilities, 1)  
  
    # Retornar os três valores: classe prevista, confiança (probabilidade) e  
probabilidades das classes  
    return predicted.item(), confidence.item(), probabilities.cpu().numpy()
```

Função encarregada de carregar imagem, fazer o pré-processamento, realizar a predição e obter o resultado com as probabilidades.

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – *Salvar Modelo Gerado, Carregar e Teste com imagem nova*

```
# Carregar o modelo salvo
model = models.resnet18(weights=ResNet18_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
model.fc = torch.nn.Linear(model.fc.in_features, 5)

# Carregar os pesos do modelo com a opção weights_only=True para evitar o aviso de segurança
model.load_state_dict(torch.load(model_path, map_location=device,
weights_only=True), strict=False)
model = model.to(device)
print(f"Modelo carregado de {model_path}")
```

Carrega o modelo salvo e os seus pesos

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – *Salvar Modelo*

Gerado, Carregar e Teste com imagem nova

```
# Obter o diretório atual de trabalho
```

```
current_directory = os.getcwd()
```

```
# Caminho para a imagem que deseja classificar
```

```
image_path = current_directory + "\\imagemMinhaFlorRosa.jpg"
```

```
# Mapear os índices para as classes (labels)
```

```
class_labels = {v: k for k, v in train_dataset.class_to_idx.items()}
```

```
# Realizar a previsão na nova imagem
```

```
predicted_class, confidence, probabilities = predict_new_image(image_path,  
model, device, test_transform)
```

```
# Obter o label associado ao número da classe
```

```
predicted_class_label = class_labels[predicted_class]
```

Obtém uma nova imagem para teste do modelo carregado, mapeia os valores dos códigos associados a cada classe para o seu rótulo, faz a previsão para a imagem obtida e pega o rótulo associado a essa previsão.

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – *Salvar Modelo*

Gerado, Carregar e Teste com imagem nova

```
# Exibir o nome da classe prevista e a confiança
print(f"Classe prevista: {predicted_class_label}")

# Mostrar a imagem com a classe prevista e a probabilidade
image = Image.open(image_path)

# Plotando a imagem e as informações previstas
plt.figure(figsize=(5, 5))
plt.imshow(image)
plt.axis('off') # Desativa os eixos
plt.title(f"Rótulo: {predicted_class_label}\nClasse:
{predicted_class}\nProbabilidade: {confidence:.4f}")
plt.show()
```

Mostra a imagem e a previsão

Convolutional Neural Networks

Exemplo – PyTorch - *Transfer Learning* com ResNet18 – *Salvar Modelo*

Gerado, Carregar e Teste com imagem nova

```
print(f"A classe prevista para a imagem é: rótulo: {predicted_class_label},  
código: {predicted_class}")  
  
print(f"Probabilidade da classe prevista: {confidence:.4f}")  
  
# Mostrar as probabilidades de todas as classes  
for i, prob in enumerate(probabilities[0]):  
    print(f"Rótulos: {class_labels[i]} => Classe {i}: Probabilidade =  
{prob.item():.4f}")
```

Apresenta o rótulo e a probabilidade da classe prevista para a imagem, depois mostra os valores das probabilidades que o modelo carregado previu para todas as classes de flores.

Convolutional Neural Networks

Exemplo – Comparação

Etapa	TensorFlow/Keras	PyTorch
Modelo base pré-treinado	MobileNetV2	ResNet18
<i>Dataset</i>	tf_flowers (via tfds)	ImageFolder com 5 pastas por classe
Congelamento inicial	base_model.trainable = False	param.requires_grad = False
<i>Fine-tuning</i>	base_model.trainable = True	param.requires_grad = True
Otimizador	Adam(lr=1e-3 → 1e-5)	Adam(lr=1e-3 → 1e-5)
Função de perda	categorical_crossentropy	CrossEntropyLoss()
Número de épocas	5 + 3	5 + 3
Salva modelo gerado		torch.save(model.state_dict(), model_path)
Carrega o modelo e faz previsão		model = models.resnet18(weights=ResNet18_Weights.DEFAULT) e output = model(image)

Bibliografia

BÁSICA:

- AGGARWAL, Charu C. **Artificial Intelligence: A Textbook**. New York: Springer: 2021.
- CHOLLET, François. **Deep Learning with Python, 2ed**. Shelter Island: Manning, 2021.
- GÉRON, Aurélien. **Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems**, 2 ed. Sebastopol: O'Reilly, 2019.

COMPLEMENTAR:

- GOODFELLOW, Ian; BENGIO, Yoshua, COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016.
- RASCHKA, Sebastian; MIRJALILI, Vahid. **Python Machine Learning**. 3 ed. Birmingham: Packt, 2017.
- RUSSEL, Stuart; NORVIG, Peter. **Artificial Intelligence: A Modern Approach**. 3 ed. Upper Saddle River: Pearson, 2010.
- TAN, Pang-Ning; STEINBACH, Michael; KUMAR, Vipin. **Introduction to Data Mining**. 2 ed. Upper Saddle River: Pearson, 2018.
- VANDERPLAS, Jake. **Python Data Science Handbook**. Sebastopol: O'Reilly, 2017.

ADICIONAIS:

- FACELI, Katti et al. **Inteligência artificial: uma abordagem de aprendizado de máquina**. 2ª Ed. Rio de Janeiro: LTC- Livros Técnicos e Científicos, 2021.
- LUGER, George F. **Inteligência Artificial** - 6ª ed. São Paulo: Pearson Education do Brasil, 2015.
- RUSSELL, Stuart; NORVIG, Peter. **Inteligência artificial: Uma Abordagem Moderna** - 4ª. Ed. GEN LTC, 2022.